

DSA: Beginner to Advanced Roadmap

A comprehensive checklist of Data Structures & Algorithms topics, from fundamentals to advanced competitive-programming concepts.

Recommended Levels

Level	Main Goal	Typical Coverage
Beginner	Build core problem-solving ability	Foundations → Arrays → Strings → Searching → Sorting → Linked Lists → Stack/Queue
Intermediate	Interview-ready DSA	Hashing → Patterns → Trees/BST → Heap → Backtracking → Greedy → Graphs → Basic
Advanced	High-level product/competitive interviews	Advanced graphs → Advanced DP → Trie/string algorithms → Segment/Fenwick trees →

How to Use This Roadmap

Study topics in order, but do not wait for perfection before solving problems. For each topic: learn the concept → implement it in Java → solve Easy problems → solve Medium problems → review patterns → revisit mistakes.

For an intermediate interview target, prioritize Arrays, Strings, Hashing, Binary Search, Linked Lists, Stack/Queue, Trees, BST, Heap, Graph BFS/DFS, Greedy, Backtracking and core DP. Advanced topics can be added later based on the companies and roles you target.

1. Foundations & Complexity

- Programming basics: variables, loops, conditionals, functions, input/output
- Time complexity: Big-O, Big-Theta, Big-Omega; best/average/worst case
- Space complexity and auxiliary space
- Amortized analysis: basic idea
- Recursion fundamentals and call stack
- Iterative vs recursive solutions
- Java Collections Framework: ArrayList, LinkedList, HashMap, HashSet, TreeMap, TreeSet, ArrayDeque, PriorityQueue

2. Arrays

- Traversal, insertion, deletion, update, searching
- Reversal, rotation, rearrangement
- Frequency counting and duplicate detection
- Prefix sum, suffix sum, difference array
- Subarrays and subsequences
- Kadane's algorithm
- 2D arrays and matrices: traversal, rotation, spiral traversal, search
- Merge arrays and sorted arrays
- Common patterns: two pointers, sliding window, prefix sum

3. Strings

- String traversal and manipulation
- StringBuilder / mutable strings

- Frequency counting
- Palindrome and anagram problems
- Substrings and subsequences
- Longest/shortest substring patterns
- String hashing: basic
- Pattern matching: naive approach
- KMP / Z algorithm: advanced

4. Searching

- Linear search
- Binary search
- First and last occurrence
- Lower bound / upper bound
- Search in rotated sorted array
- Peak element
- Binary search on answer / search space
- Advanced binary-search patterns

5. Sorting

- Bubble sort, selection sort, insertion sort
- Merge sort
- Quick sort
- Heap sort
- Counting sort
- Radix sort
- Bucket sort
- Cyclic sort
- Stability, in-place sorting, comparison vs non-comparison sorting
- Sorting-based problem-solving patterns

6. Hashing

- Hash tables and hashing concepts
- HashMap / HashSet usage
- Frequency maps
- Two Sum pattern
- Grouping and counting
- Prefix sum + hashing
- Longest consecutive sequence
- Collision concept and basic hash design

7. Linked Lists

- Singly linked list
- Doubly linked list
- Circular linked list
- Insert/delete/search
- Reverse linked list
- Middle node
- Fast and slow pointers
- Cycle detection and removal
- Merge sorted lists
- Remove Nth node
- Intersection of lists
- Linked-list sorting and advanced pointer manipulation

8. Stack

- Stack implementation using arrays/linked lists
- Java Stack / Deque
- Balanced parentheses
- Prefix, infix, postfix expressions
- Expression conversion and evaluation
- Next greater/smaller element
- Monotonic stack
- Histogram / stock-span style problems

9. Queue, Deque & Priority Queue

- Queue implementation
- Circular queue
- Deque
- Queue using stacks / stack using queues
- BFS fundamentals
- Priority queue
- Min heap / max heap usage
- Top-K patterns

10. Recursion & Backtracking

- Base case and recursive case
- Recursion trees
- Parameter/state passing
- Subsets and subsequences
- Permutations
- Combinations
- Combination Sum

- Backtracking: choose, explore, undo
- N-Queens
- Rat in a Maze
- Sudoku: advanced backtracking

11. Binary Trees

- Tree terminology: root, leaf, height, depth, level
- Preorder, inorder, postorder
- Recursive and iterative traversals
- Level-order / BFS traversal
- Maximum/minimum depth
- Count/sum nodes
- Same tree and symmetric tree
- Invert tree
- Balanced tree
- Diameter
- Path Sum
- Lowest Common Ancestor
- Left/right/top/bottom views
- Vertical traversal
- Boundary traversal
- Serialization/deserialization

12. Binary Search Trees

- BST properties
- Search, insertion, deletion
- Minimum and maximum
- Inorder predecessor/successor
- Validate BST
- Lowest Common Ancestor
- Kth smallest/largest
- Convert sorted array/list to BST
- Recover BST
- Balanced BST concepts

13. Heaps & Heap Algorithms

- Min heap and max heap
- Heapify
- Insert/delete/extract
- Heap sort
- Priority queue

- Kth largest/smallest
- Top K frequent/elements
- Merge K sorted lists
- Two-heap median pattern
- Advanced heap applications

14. Greedy Algorithms

- Greedy-choice property and proof intuition
- Activity selection
- Fractional knapsack
- Assign Cookies
- Jump Game
- Gas Station
- Job sequencing
- Interval scheduling
- Merge intervals
- Non-overlapping intervals
- Huffman coding: advanced

15. Intervals

- Sort intervals by start/end
- Merge intervals
- Insert interval
- Interval intersection
- Meeting Rooms I/II
- Non-overlapping intervals
- Sweep-line concept
- Difference-array / event-based interval techniques

16. Graph Fundamentals

- Vertices, edges, degree
- Directed vs undirected
- Weighted vs unweighted
- Connected components
- Adjacency matrix
- Adjacency list
- BFS
- DFS
- Visited arrays/sets
- Grid graphs and matrix traversal
- Cycle detection

17. Graph Algorithms

- Topological sort: DFS and Kahn's algorithm
- Course Schedule pattern
- Shortest path in unweighted graphs
- Dijkstra's algorithm
- Bellman-Ford
- Floyd-Warshall
- Minimum Spanning Tree: Kruskal and Prim
- Disjoint Set Union (Union-Find)
- Path compression
- Union by rank/size
- Bipartite graphs
- Strongly Connected Components: Kosaraju / Tarjan
- Bridges and articulation points: advanced

18. Dynamic Programming

- DP mindset: state, transition, base case
- Memoization vs tabulation
- 1D DP: Fibonacci, Climbing Stairs, House Robber
- Grid DP: Unique Paths, Minimum Path Sum
- 0/1 Knapsack
- Unbounded Knapsack
- Subset Sum / Partition
- Coin Change
- Longest Common Subsequence
- Longest Increasing Subsequence
- Edit Distance
- Palindromic subsequence/string DP
- Interval DP
- Tree DP
- Bitmask DP: advanced
- Digit DP: advanced

19. Bit Manipulation

- Binary representation
- AND, OR, XOR, NOT
- Left/right shifts
- Set, clear, toggle, test a bit
- Odd/even and power-of-two checks
- Count set bits

- XOR tricks
- Single Number patterns
- Bitmasking
- Subset generation with bitmasks
- Bitmask DP: advanced

20. Trie & String Data Structures

- Trie structure
- Insert/search/startsWith
- Prefix queries
- Word Dictionary
- Autocomplete concepts
- Compressed trie / radix tree: advanced
- KMP and Z algorithm
- Suffix array / suffix tree: advanced

21. Divide & Conquer

- Divide, solve, combine
- Recurrence relations
- Binary search
- Merge sort
- Quick sort
- Counting inversions
- Closest pair / advanced divide-and-conquer concepts

22. Mathematical Algorithms

- GCD and LCM
- Euclidean algorithm
- Prime checking
- Sieve of Eratosthenes
- Prime factorization
- Modular arithmetic
- Fast exponentiation
- Combinatorics basics
- Modular inverse: advanced
- Matrix exponentiation: advanced

23. Advanced Data Structures

- Segment Tree
- Lazy Propagation
- Fenwick Tree / Binary Indexed Tree

- Sparse Table
- Interval Tree
- Ordered sets / balanced BST concepts
- Treap
- Skip List
- Heavy-Light Decomposition
- Link-Cut Tree: advanced

24. Advanced Graph & Optimization

- Max Flow / Min Cut
- Ford-Fulkerson
- Edmonds-Karp
- Dinic's algorithm
- Min-cost flow: advanced
- Matching: bipartite matching
- Euler path/circuit
- Hamiltonian path: advanced/NP-complete context
- Network flow applications

25. Advanced String & Computational Topics

- Suffix array
- Suffix tree
- Suffix automaton
- Rolling hash
- Rabin-Karp
- Advanced KMP/Z applications
- Computational geometry basics
- Convex hull
- Line sweep
- Closest pair of points

26. Advanced DP & Competitive Programming

- Bitmask DP
- Digit DP
- Tree DP
- Rerooting DP
- DP optimization
- Divide-and-conquer DP optimization
- Convex Hull Trick
- Meet-in-the-middle
- State compression

- Game DP / minimax concepts

27. Problem-Solving Patterns

- Two pointers
- Sliding window
- Prefix/suffix sums
- Hashing
- Fast/slow pointers
- Binary search on answer
- Monotonic stack/queue
- Top K with heap
- Intervals
- DFS/BFS
- Backtracking
- Greedy
- Union-Find
- Topological sorting
- 1D/2D DP
- Tree recursion
- Graph shortest-path patterns

28. Interview & Competitive Programming Skills

- Read constraints before choosing an algorithm
- Estimate time and memory before coding
- Write brute force first, then optimize
- Identify patterns from examples
- Edge-case analysis
- Proof / correctness intuition
- Clean implementation and naming
- Testing and debugging
- Explain time and space complexity
- LeetCode-style interview practice
- Competitive programming: contests, speed, implementation under pressure

Suggested Completion Milestones

Milestone	What to be able to do
Beginner complete	Solve common Easy problems, use basic data structures, analyze Big-O, and implement standard algorithms.
Intermediate complete	Solve most common Easy and many Medium problems, recognize patterns, handle trees/graphs/heap, and ex
Advanced complete	Handle difficult Medium/Hard problems, advanced DP/graphs/data structures, and optimize solutions under co